AI-first Software Engineering Program - Case Study

Relay: AI Workflow Orchestrator

Author : **Airtribe**

BACKGROUND & OBJECTIVE

Operations teams run on workflows. When an order webhook arrives, a system may need to check the payload, classify the request, notify the right team, wait for approval, and then take an action.
In AI-first products, workflows become more complex because some steps now involve machine judgment. An AI node may read a webhook payload, classify the request, and decide which branch the workflow should take. But this only works if the orchestration engine is reliable underneath.
This project involves designing and implementing Relay, an AI workflow orchestration platform that can define workflows as connected typed nodes, trigger runs from webhooks or APIs, execute steps durably in the background, pause for human approvals, validate AI outputs, recover from crashes, and maintain complete run traces.

KEY FEATURES

1. Workflow Definitions:
Allow users to create, update, list, and publish workflows made of typed nodes. Workflows should follow a draft → publish lifecycle. Published workflows should be frozen and runnable.

2. Webhook and Manual Triggers:
Support webhook-based triggers protected by a workflow secret, along with manual trigger APIs for testing and demos. The trigger payload becomes the input for the run.

3. Durable Workflow Execution:
Execute runs asynchronously through a queue and worker. Every step should be persisted so that if the worker crashes, the run can resume from the last completed step instead of starting again.

4. Deterministic and AI Nodes:
Implement deterministic node types such as http_request, condition, delay, and notify. Also implement an ai node where the model output is validated against a JSON schema before downstream steps can use it.

5. Approval Gates and Sensitive Actions:
Implement approval nodes where a run pauses until a human approves or rejects it. Sensitive nodes, such as order actions, must not execute unless an approval record exists. This should be enforced by the engine, not by the AI prompt.

6. Traces, Guardrails and Step Cap:
Store a complete trace for every run, including resolved inputs, outputs, attempts, timing, and token usage for AI steps. Handle prompt-injection payloads safely, prevent duplicate side effects with idempotency keys, and enforce a max-step cap so runaway workflows cannot loop forever.

TECHNICAL REQUIREMENTS

- Expose the core functionality as RESTful APIs.
- Use a reliable persistent store for workflows, runs, run definition snapshots, steps, and approvals.
- Execute workflows asynchronously using a durable queue and worker loop.
- Side-effect calls to the mock world must use idempotency keys so crash recovery does not duplicate actions.
- Every external call, including model providers and mock-world calls, should have a timeout.
- Keep the AI provider behind an adapter so it can be mocked in engine tests.

- Enforce approval gates and the step cap in the engine, independent of model output.
- Provide automated tests for publish validation, template resolution, schema enforcement, idempotency, and guardrails.

ASSESSMENT CRITERIA

- Functionality:
  Does the platform load the provided data, manage draft and published workflows, create per-run definition snapshots, trigger runs from webhooks and manual APIs, execute deterministic and AI nodes, pause for approvals, enforce the step cap, and show everything through a simple console?
- Durability and AI Correctness:
  Does a killed worker resume without duplicating side effects? Do injection payloads fail to bypass approval gates? Does schema enforcement catch malformed AI outputs? Does the runaway workflow stop at the step cap?
- Code Quality:
  Is the code clean, organized, maintainable, and easy to understand?
- Design and Structure:
  Is the orchestration engine well-designed? Does it show a good understanding of queues, workers, state machines, idempotency, retries, durable execution, AI nodes, and approval-gated systems?
- Documentation and Demo:
  Is the README clear? Are setup steps, API documentation, architecture, run state machine, recovery design, and known limitations documented? Does the demo clearly show the full Relay workflow?

DELIVERABLES

- The final, functional product: orchestration engine and simple console.
- README file covering setup instructions, API documentation, architecture, design decisions, run state machine, and exactly-once recovery approach.
- Public link of the GitHub repository.
- Seeded demo workflows and instructions to reproduce the demo using the mock world.
- Verification report with smoke test output and duplication check result after a live kill-and-resume demo.
- Explainer video demonstrating the project, including workflow trigger, AI node branching, approval gate, worker crash and resume, prompt-injection handling, and step-cap protection.

RESOURCES

GitHub Repository:
https://github.com/airtribe-projects/relay-capstone